

Hello, Distributed System

BEGINNER TECHNICAL PLAYBOOK • 80/20 + FEYNMAN • EVIDENCE FROM THE UPLOADED ARCHIVE

LAB 02

Node.js processes

TCP 127.0.0.1:4000

current logs/results are primary evidence

Start Here: What Is This?

MUST KNOW This lab runs **two independent Node.js processes** on one Windows machine. They talk over a real TCP socket at 127.0.0.1:4000.

It demonstrates three ideas: **latency**, **partial failure**, and **process independence**. It does *not* simulate separate machines, routing, or a production network.

Why Does It Exist?

Instead of only reading definitions, the lab makes you **observe** them. You run normal traffic, add a server-side delay, kill the server during work, then inspect logs and focused verification tests.

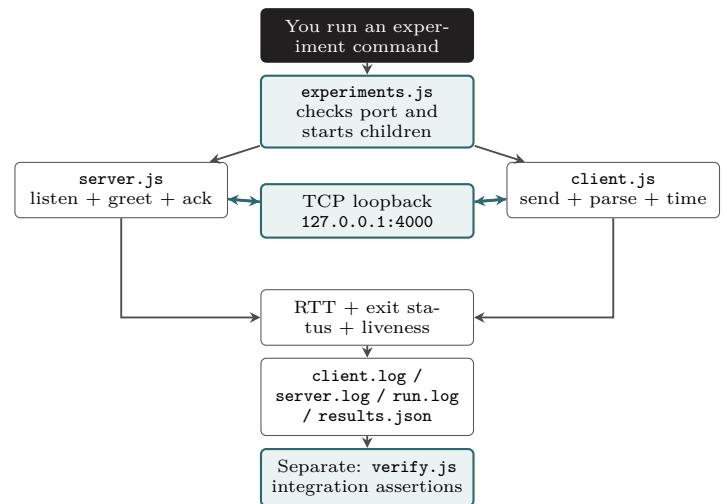
The 80/20 core

- `server.js` listens; `client.js` connects.
- A newline marks one complete message frame.
- The client times request → acknowledgment as RTT.
- Latency mode adds **200 ms before each acknowledgment**.
- Chaos mode kills only the server after the first acknowledgment.

Main Components

Component	Beginner meaning	Actual job
<code>server.js</code>	service side	Listens, greets, parses lines, sends JSON acks.
<code>client.js</code>	requester side	Sends text, parses JSON, measures RTT, handles errors/timeouts.
<code>experiments.js</code>	conductor	Starts processes, changes delay, kills server in chaos, writes results.
<code>chaos.ps1</code>	launch button	Thin PowerShell wrapper for <code>experiments.js</code> <code>chaos</code> .
<code>verify.js</code>	test inspector	Runs real/fake TCP peers and assertions; writes PASS/FAIL log.

Whole Project - One Trace



If you remember only one thing

The lab compares the same client/server conversation under normal, delayed, and deliberately broken server conditions, then uses logs to show what changed.

Folder Map: What Matters

```
sd-lab-02/
|-- server.js      <- receives lines; sends JSON
|-- client.js      <- sends; measures RTT
|-- experiments.js <- experiment harness
|-- chaos.ps1      <- PowerShell chaos wrapper
|-- verify.js      <- focused TCP checks
|-- README.md      <- reproduce/usage notes
|-- lab-report.md  <- narrative (some stale values)
|-- terminal_cmds.txt <- recorded commands/output
`-- logs/
    |-- baseline/   <- normal run evidence
    |-- latency/    <- 200 ms delay evidence
    |-- chaos/      <- forced server failure
    |-- chaos-custom/ <- one custom-message chaos run
    |-- verification.log
    |-- verification-first-attempt.log
    `-- final-check.log
```

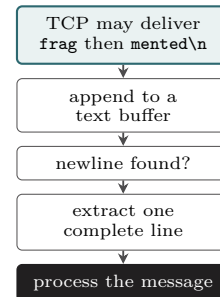
Each experiment folder contains `server.log`, `client.log`, `run.log`, and `results.json`.

Beginner Decoder

Process	One running program with its own PID, memory, timers, and event loop.
Client	The side that connects and asks for work.
Server	The side that listens and replies.
TCP	A reliable byte stream between the two processes.
Port 4000	The numbered local endpoint where the server listens.
127.0.0.1	Loopback: this same computer, not the Internet.
JSON	Structured text used for server replies.
Log	A time-ordered record of what happened.
RTT	Round-trip time: send request to parsed acknowledgment.
PID	Operating-system process identifier.

The Message Rule: One Line = One Frame

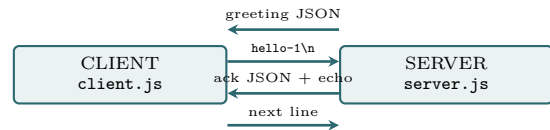
MUST KNOW TCP gives bytes, not ready-made messages. Both programs therefore collect bytes until they see `\n`.



Feynman version

Simple idea: wait until the sentence ends.
Analogy: a cashier waits for the full order slip, not half a word.
Project reality: `server.js` and `client.js` keep incomplete text in buffer.
Technical term: newline-delimited framing over a TCP byte stream.

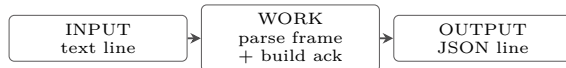
Actual Protocol



Server replies are JSON objects such as a greeting `{type,pid,counter}` and acknowledgments that also include `echo`.

server.js - The Listener

Why it exists. It owns port 4000, accepts a client, sends the greeting, turns each complete input line into an acknowledgment, and optionally delays that acknowledgment.



Only important code

```
server.listen(4000, '127.0.0.1');
const ack = { type: 'ack', pid: process.pid,
               counter: ++counter, echo: line };
if (delay === 0) send(ack);
else setTimeout(() => send(ack), delay);
```

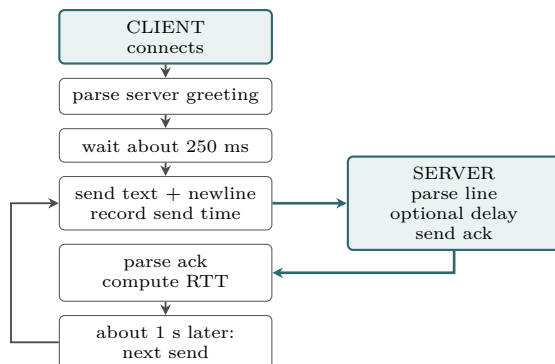
Plain English: listen locally; echo each line back in structured form; wait first only when a delay is configured.

Why it matters: one environment variable creates the baseline/latency difference without changing source code.

Timeout behavior

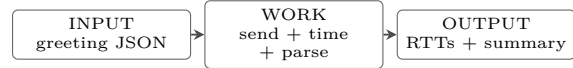
Each connected socket has a **5 s idle timeout**. On inactivity the server destroys that connection, but the listener process itself keeps running until the harness stops it.

Client <-> Server Flow



client.js - The Measurer

Why it exists. It connects, waits for the greeting, sends either three default messages or one custom message, measures RTT, and reports how the connection ended.



Only important code

```
sent.set(text, performance.now());
socket.write(text + '\n');
...
const rtt = performance.now() - sent.get(message.echo);
rtts.push(rtt);
```

Plain English: save the clock just before sending; subtract that saved time when the matching acknowledgment arrives.

Why it matters: this is the measurement used in `results.json`.

Two different safety timers

Idle timeout = 5 s. Fires when no bytes arrive. A handled idle close leaves exit code 0.

Hard timeout = 8 s. Absolute deadline. It catches a peer that keeps sending incomplete bytes and therefore prevents the idle timer from firing; exit code becomes 1.

What RTT Means Here

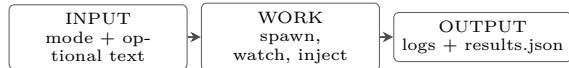
$$\text{RTT} = t_{\text{ack parsed}} - t_{\text{request sent}}$$

In normal words: how long the client waits from sending a line until it successfully parses the matching acknowledgment.

It includes local write/logging overhead, TCP loopback transport, server work, intentional `setTimeout`, scheduling, and client parsing. It is **not** network-only latency.

experiments.js - The Conductor

Why it exists. It gives every experiment the same controlled start/stop procedure and collects evidence.



The two experiment switches

```
const delay = mode === 'baseline' ? 0 : 200;
...
if (mode === 'chaos') {
  await sleep(100);
  server.child.kill('SIGKILL');
}
```

Plain English: baseline replies immediately; latency and chaos delay replies by 200 ms; chaos additionally kills the server 100 ms after the first ack is observed.

Why it matters: the real failure injection is here, not inside chaos.ps1.

chaos.ps1 - What It Really Does

It is a **thin PowerShell launcher**. With no argument it runs:

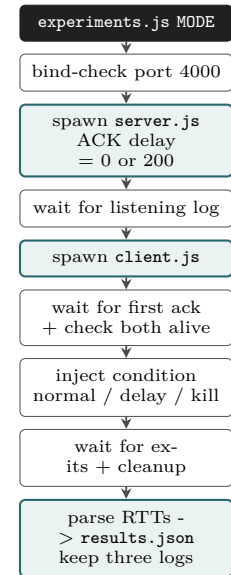
```
node experiments.js chaos
```

With **-Message**, it forwards one custom text argument. It does **not** implement the kill itself.

Execution Order

1. Check that port 4000 can be bound.
2. Start a fresh server with delay 0 or 200 ms.
3. Wait until its log says it is listening.
4. Start the client.
5. Wait until the first acknowledgment appears.
6. Confirm both PIDs are alive together.
7. Chaos only: wait about 100 ms, then kill the server.
8. Wait for client exit; normal modes require server survival.
9. Stop surviving server, parse RTTs from `client.log`, write `results.json`.

Experiment Pipeline

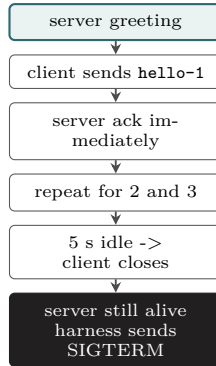


What the harness protects against

Readiness, first-ack, client-completion, and server-completion waits are bounded. A busy port aborts the run; the harness never kills an unrelated listener. Cleanup targets only children that it created.

Baseline: Nothing Intentionally Slowed

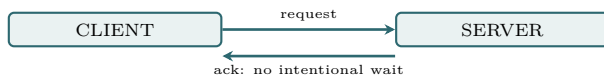
MUST KNOW ACK_DELAY_MS=0.



Observed current baseline

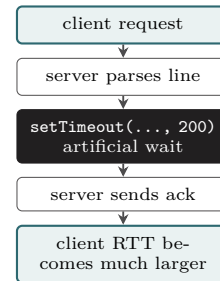
RTTs: **1.007, 0.795, 0.530 ms**.
Mean: **0.777 ms**.
Acknowledged: **3/3**. Client exit: **0**.
Server remained alive after client exit: **true**.

Baseline Visual



Latency: Same Conversation + 200 ms Wait

MUST KNOW ACK_DELAY_MS=200. The delay is inside `server.js`; it is **application delay**, not simulated physical network delay.



Observed current latency run

RTTs: **216.736, 201.395, 216.847 ms**.
Mean: **211.659 ms**.
Acknowledged: **3/3**. Client exit: **0**.
Server remained alive after client exit: **true**.

What Changed?

$$211.659 - 0.777 = \mathbf{210.882\ ms}$$

The mean RTT increased by **210.882 ms** in these current runs. The added code wait is 200 ms; scheduling and other local work explain the extra variation.

Only three samples were collected per normal run. Treat this as a **demonstration**, not a statistical benchmark.

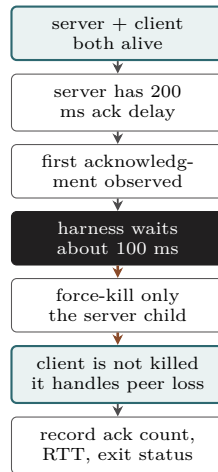
Chaos: Intentionally Remove One Process

Memorable analogy

Imagine two people on a phone call. After the first successful reply, an experimenter abruptly removes the server's phone. The client is still running, but the conversation is broken.

Map it to the project: people = processes; phone line = TCP connection; first reply = `hello-1` acknowledgment; removed phone = forced server termination; call error = client socket event.

Chaos Flow



Observed current default chaos run

First RTT: **201.534 ms**. Then: **ECONNRESET**.
Acknowledged: **1/3**. Client exit: **1**.
Server exit status: `code=null`, `signal=SIGKILL`.
The client never sent `hello-2` or `hello-3` after the reset.

Why Break It On Purpose?

Simple idea: learn how one part reacts when another disappears.

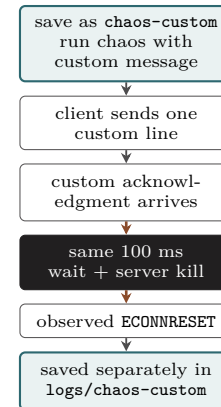
Project reality: the server dies while the client process is still active.

Technical term: a controlled **partial-failure** / **chaos experiment**.

The important evidence is not simply “something failed.” It is that **one process failed first**, while the other detected the broken connection and ran its own cleanup/error path.

The harness intends to remove the server. The exact client-side symptom may be `FIN`, `ECONNRESET`, or another socket error depending on OS/timing. This Windows run observed `ECONNRESET`.

Custom Chaos: Different Payload



Observed custom run

Payload: **custom payload with spaces**.

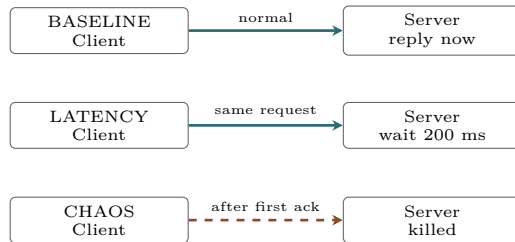
RTT: **205.107 ms**; acknowledged **1/1**; then **ECONNRESET**; client exit **1**.

Important nuance: all intended payload work completed, yet the later socket error still made the client exit 1.

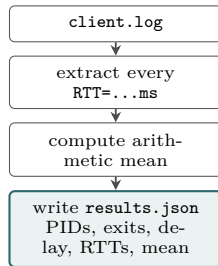
Experiments: Expected vs Observed Evidence

Experiment	What changes?	Why test it?	Expected behavior	Current observed evidence
Baseline	Ack delay 0 ms	Reference behavior	Three quick acks; client can finish while server listener survives	RTTs 1.007/0.795/0.530; mean 0.777 ms; 3/3; client 0; server alive then SIGTERM
Latency	Ack delay 200 ms	Make delay visible	RTT rises roughly with server-side wait	216.736/201.395/216.847; mean 211.659 ms; 3/3; client 0
Chaos	200 ms delay + kill server after first ack	Show partial failure	Peer disappears; client must react; exact socket symptom can vary	first RTT 201.534 ms; ECONNRESET; 1/3; client 1; server SIGKILL
Custom chaos	Same chaos path, one custom message; separate log folder	Prove custom-message path is matched correctly	Wait for actual custom ack, then kill	205.107 ms; custom ack 1/1; ECONNRESET; client 1

One Simple Comparison



How results.json Is Made



The acknowledgment count and error reason are easiest to read in `client.log`; they are not explicit fields in `results.json`.

Evidence Discrepancy You Should Know

`lab-report.md` contains **older baseline, latency, and default-chaos PIDs/RTTs** from earlier runs. The current experiment logs and JSON were rerun later and contain different values. The custom-chaos values still match.

This playbook therefore uses **current results.json + current logs** for all experiment numbers. The report remains useful for concepts, but not for those stale measurements.

The verification section of the report also lists older PIDs than the current `verification.log`. The PASS behaviors are consistent; the IDs are not.

What This Lab Does NOT Prove

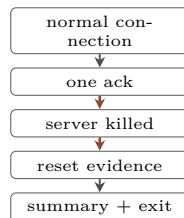
- It does not prove behavior across two physical machines.
- It does not test routing, network partitions, packet loss, or the Internet.
- It does not provide a statistical latency benchmark (only three normal samples per run).
- It does not guarantee ECONNRESET on every OS/run.
- Linux commands are documented, but this archive says they were not executed because WSL lacked Node.js.

How To Read One Useful Log

Use the current logs/chaos/client.log. Read it as a story:

Representative log lines

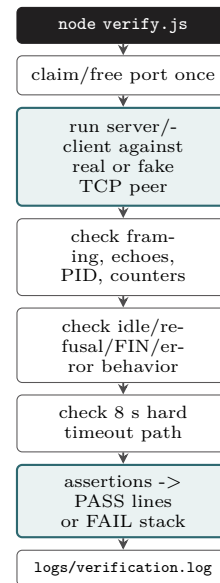
```
[client pid=29628] connected to 127.0.0.1:4000
  ↳ TCP connection exists.
[client pid=29628] ack "hello-1"; ... RTT=201.534ms
  ↳ One delayed request completed.
[client pid=29628] socket error: ECONNRESET
  ↳ The peer vanished abruptly in this run.
... acknowledged=1/3
  ↳ Two intended messages never completed.
... exit code=1
  ↳ The client classified the socket error as failure.
```



To inspect other logs, find: **start/connect** → **messages** → **change/error/timeout** → **final summary**.

verify.js - What It Actually Verifies

Critical correction: `verify.js` does **not** read `experiment results.json` and compare it to thresholds. It launches real programs and fake TCP peers, then uses assertions.



Actual Verification Conditions

- Split input: no ack until newline; then exact echoes **fragmented, second, third**.
- Counters must be 1,2,3,4 and all server messages must contain the real server PID.
- A silent peer must trigger the server's 5 s idle disconnect.
- Client with no listener: `ECONNREFUSED`, empty RTT list, exit 1.
- Fragmented greeting + batched acknowledgments: client parses all 3, exit 0 after remote FIN.
- Silent fake server: client 5 s idle timeout, exit 0.
- Continuously arriving but incomplete JSON: idle timer stays reset, so hard 8 s timeout must fire; exit 1.

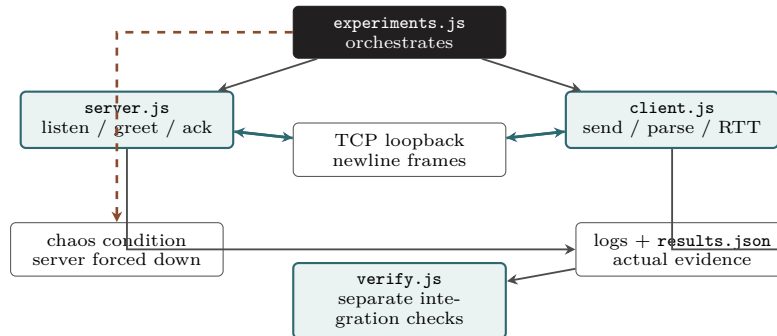
Recorded outcome

The current `verification.log` ends: **“All verification checks passed.”** The earlier `verification-first-attempt.log` failed with `spawn EPERM`; the archive identifies that as an execution-environment restriction, not a TCP behavior failure.

The Project in 60 Seconds

A Node.js server listens on local TCP port 4000. A separate client connects, receives a JSON greeting, sends newline-terminated text, and measures the time until each JSON acknowledgment returns. The experiment harness runs the same system in three conditions. Baseline sends acknowledgments immediately. Latency waits 200 ms before each acknowledgment. Chaos keeps that delay, waits for the first acknowledgment, then forcibly kills only the server. Current logs show sub-millisecond baseline RTTs, roughly 200+ ms delayed RTTs, and an `ECONNRESET` after the first chaos acknowledgment. Separate verification tests check framing, PIDs/counters, connection refusal, idle handling, fragmented JSON, and the hard deadline.

The Project in One Diagram



Remember These 8 Things

1. Client and server are different OS processes.
2. They communicate through TCP loopback, not shared objects.
3. Newline framing turns a byte stream into messages.
4. The server sends JSON; the client sends plain text lines.
5. RTT measures request-send to matching-ack parse.
6. The 200 ms latency is application code, not physical network delay.
7. Chaos kills only the server after the first ack; the client handles peer loss itself.
8. Verification tests behavior directly; it does not validate `results.json` thresholds.

Can I Explain It?

Questions

1. What does the server do?
2. Why is a baseline needed?
3. What exactly changes in latency mode?
4. What does chaos remove?
5. Where do experiment measurements end up?
6. Why is newline framing necessary?
7. What does `verify.js` prove?

Short answers

1. Listens, greets, parses lines, sends acknowledgments.
2. It gives a normal reference for comparison.
3. Server acknowledgments wait 200 ms.
4. The harness forcibly terminates its own server process.
5. Logs plus each experiment's `results.json`.
6. TCP has bytes, not inherent message boundaries.
7. Specific server/client behaviors through assertions and real TCP sockets.

Command Cheat Sheet

Commands below are supported by the uploaded project. The recorded environment used Windows PowerShell and Node.js v24.19.0; README requires Node 18+.

Purpose	Command	What it does
Check Node	<code>node -version</code>	Confirm a usable Node runtime.
Baseline	<code>node experiments.js baseline</code>	Fresh server/client; no ack delay; save baseline evidence.
Latency	<code>node experiments.js latency</code>	Fresh server/client; 200 ms ack delay.
Chaos	<code>.\chaos.ps1</code>	Run chaos mode through the PowerShell wrapper.
Chaos direct	<code>node experiments.js chaos</code>	Same core chaos workflow without the wrapper.
Custom chaos	<code>\$env:LAB_RUN_NAME='chaos-custom'; .\chaos.ps1 -Message 'custom payload with spaces'</code>	One custom message; stores it separately.
Remove temp env	<code>Remove-Item Env:LAB_RUN_NAME</code>	Return PowerShell environment to normal.
Syntax checks	<code>node -check server.js</code>	Parse-check one JS file; README repeats for client/experiments.
Verification	<code>node verify.js</code>	Run focused integration assertions and write verification log.
Manual server	<code>\$env:ACK_DELAY_MS='0'; node server.js</code>	Start server yourself in Terminal A.
Manual client	<code>node client.js</code>	Connect from Terminal B and send three defaults.
Custom client	<code>node client.js 'my message'</code>	Send one single-line custom payload.

Fast Self-Check Before Explaining the Lab

- ☐ I can draw client ↔ server over TCP.
- ☐ I can explain why newline framing exists.
- ☐ I can define RTT in one sentence.
- ☐ I can say what changes in baseline vs latency.
- ☐ I can say who kills the server in chaos.
- ☐ I can read `acknowledged=1/3`.
- ☐ I know current JSON/log values outrank stale report numbers.
- ☐ I know verification is separate from experiment result comparison.

Evidence Priority

When two files disagree, use this order for this archive:

1. **Executable source code** for what the programs actually do.
2. **Current results.json and logs** for what the latest recorded runs actually did.
3. **README/report** for intent and narrative context.

Final mental model

Normal shows the conversation works.

Latency shows time cost.

Chaos shows one side can disappear while the other side must detect and handle it.

Logs are the evidence. **Verification** checks the important edge cases.